Commit 3cecf59

Browse files

ActuallyNull committed last month

```
model 2.1, added ensemble models
```

main 1 parent 3cad777 commit 3cecf59

Filter files...

model.py

1 file changed +69 -29 lines changed

Search within code

model.py +69 -29

```
@@ -4,7 +4,7 @@
 from data_processing import create_train_val_test_splits
 from sklearn.metrics import confusion_matrix, classification_report
 import matplotlib.pyplot as plt
-from sklearn.metrics import RocCurveDisplay
+from sklearn.metrics import RocCurveDisplay, auc, precision_recall_curve
 from sklearn.preprocessing import label_binarize
 from sklearn.utils.class_weight import compute_class_weight
 from keras import regularizers
@@ -52,6 +52,14 @@ def residual_block(x, filters, kernel_size=15, strides=1):
     x = layers.ReLU()(x)
     return x

+def squeeze_excite_block(input_tensor, ratio=16):
+    filters = input_tensor.shape[-1]
+    se = layers.GlobalAveragePooling1D()(input_tensor)
+    se = layers.Dense(filters // ratio, activation='relu')(se)
+    se = layers.Dense(filters, activation='sigmoid')(se)
+    se = layers.Reshape((1, filters))(se)
+    return layers.multiply([input_tensor, se])
+
 def ecg_cnn(input_length, num_classes=3): # 0: afib, 1: normal, 2: other arrhythmia
     inputs = layers.Input(shape=(input_length, 1))
@@ -61,10 +69,15 @@ def ecg_cnn(input_length, num_classes=3): # 0: afib, 1: normal, 2: other arrhyth
     x = layers.MaxPooling1D(pool_size=3, strides=2, padding='same')(x)

     x = residual_block(x, 64)
+    x = squeeze_excite_block(x)
     x = residual_block(x, 64)
+    x = squeeze_excite_block(x)
     x = residual_block(x, 128, strides=2)
+    x = squeeze_excite_block(x)
     x = residual_block(x, 128)
+    x = squeeze_excite_block(x)
     x = residual_block(x, 256, strides=2)
+    x = squeeze_excite_block(x)
     x = residual_block(x, 256)

     x = layers.GlobalAveragePooling1D()(x)
@@ -76,41 +89,6 @@ def ecg_cnn(input_length, num_classes=3): # 0: afib, 1: normal, 2: other arrhyth
     model = models.Model(inputs, outputs)
     return model

-model = ecg_cnn(input_length=3000) # 300 hz for 10 sec = 3000 samples
-
-model.compile(optimizer='adam',
-              loss=SparseCategoricalFocalLoss(gamma=2),
-              metrics=['accuracy'])
-
-model.fit(
-    train_gen,
-    epochs=100,
-    callbacks=callback,
-    class_weight=class_weights,
-    validation_data=val_gen
-)
-
 test_gen = ECGDataGenerator(X_test, y_test, batch_size=64, augmentor=None, shuffle=False)
-test_loss, test_acc = model.evaluate(test_gen)
-print(f"Test accuracy: {test_acc:.4f}")
-
-y_pred = model.predict(test_gen)
-y_pred_classes = y_pred.argmax(axis=1)
-
-print("Confusion Matrix:")
-print(confusion_matrix(y_test, y_pred_classes))
-
-print("Classification Report:")
-print(classification_report(y_test, y_pred_classes))
@@ -94,6 +72,83 @@ print(classification_report(y_test, y_pred_classes))
+n_models = 3
+ensemble_preds = np.zeros((len(y_test), 3))  # 3 = num_classes
+ensemble_test_acc = 0
+
+for seed in range(n_models):
+    print(f"\nTraining ensemble model {seed+1}/{n_models}")
+    tf.keras.utils.set_random_seed(seed)
+    model = ecg_cnn(input_length=3000)
+    model.compile(
+        optimizer='adam',
+        loss=SparseCategoricalFocalLoss(gamma=2),
+        metrics=['accuracy']
+    )
+    model.fit(
+        train_gen,
+        epochs=100,
+        callbacks=callback,
+        class_weight=class_weights,
+        validation_data=val_gen,
+        verbose=0  # Suppress output for brevity
+    )
+
+    test_loss, test_acc = model.evaluate(test_gen)
+    ensemble_test_acc += test_acc
+    # Predict on test set
+    y_pred = model.predict(test_gen)
+    ensemble_preds += y_pred
+
+ensemble_preds /= n_models
+ensemble_pred_classes = ensemble_preds.argmax(axis=1)
+
+ensemble_test_acc /= n_models
+print(f"Average test accuracy: {ensemble_test_acc:.4f}")
+
+print("\nEnsemble Confusion Matrix:")
+print(confusion_matrix(y_test, ensemble_pred_classes))
+print("Ensemble Classification Report:")
+print(classification_report(y_test, ensemble_pred_classes))
+
+# ROC and PR curves for ensemble
 y_test_binary = label_binarize(y_test, classes=[0,1,2])
-for i in range(y_pred.shape[1]):
-    RocCurveDisplay.from_predictions(y_test_binary[:,i], y_pred[:,i], name=f"Class {i}")
+for i in range(ensemble_preds.shape[1]):
+    RocCurveDisplay.from_predictions(y_test_binary[:,i], ensemble_preds[:,i], name=f"Class {i}")

 plt.plot([0,1], [0,1], 'k--')
 plt.xlabel("FPR (False Positive Rate)")
 plt.ylabel("TPR (True Positive Rate)")
-plt.title("OvR ROC Curve")
+plt.title("Ensemble OvR ROC Curve")
+plt.legend()
+plt.show()
+
+plt.figure(figsize=(8, 6))
+for i in range(ensemble_preds.shape[1]):
+    precision, recall, _ = precision_recall_curve(y_test_binary[:, i], ensemble_preds[:, i])
+    auc_score = auc(recall, precision)
+    plt.plot(recall, precision, label=f'Class {i} (AUC = {auc_score:.2f})')
+
+plt.xlabel('Recall')
+plt.ylabel('Precision')
+plt.title('Ensemble Precision-Recall Curve (OvR)')
 plt.legend()
 plt.show()
```

Comments 0

Please sign in to comment.